Project - Low Level Design

on

AI-Driven
Construction Documentation System

Course Name: GenAI

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1.	Vivek Sharma	EN22ME304116
2.	Jayesh Chouhan	EN23MEEL4001
3.	Sakina Modi	EN22EL301044
4.	Vinay Birla	EN22CS3011083
5.	Dev Singh Ghuraiya	EN22ME304022

Group Name:05D10

Project Number:DO-18

Industry Mentor Name:

University Mentor Name: Divya Kumawat

Academic Year:2025-26

Table of Contents

1. Introduction.
   1.1. Scope of the document.
   1.2. Intended Audience
   1.3. System overview.
2. System Design.
   2.1. Architecture Diagram (ASCII)
   2.2. Class Diagram (ConstructionRAG, Pydantic models, SavedReport)
   2.3. Sequence Diagram — Report Generation
   2.4. Sequence Diagram — Chat with AI
   2.5. Activity Diagram — Report Generation Flow
3. Application Design
   3.1. Process Flow
   3.2. Information Flow
   3.3. Components Design
4. LLM Selection & Rationale
5. Approach Toward Solving the Problem Statement
6. References

1. Introduction

ConstructionGen is an **AI-powered construction documentation platform** that enables construction professionals to generate structured **site reports** and interact with a domain-specific **AI assistant**. The system combines a **Retrieval-Augmented Generation (RAG) pipeline** with a modern **React frontend** and a **FastAPI backend** to deliver accurate, grounded, and professionally formatted construction reports.

1.1 Scope of the Document

This **Low Level Design** document covers:

- The **internal architecture** and **component breakdown** of the ConstructionGen system
- **UML diagrams** (Class, Sequence, Activity) representing **system behaviour**
- The overall **system architecture diagram**
- The **LLM selection rationale**
- The **approach** taken to solve the core **problem statement**
- **API contracts** between frontend and backend
- **Key design decisions** made during development

This document does not cover **deployment infrastructure**, **CI/CD pipelines**, or **user authentication systems**.

1.2 Intended Audience

Software Developers – Engineering personnel responsible for the long-term maintenance or programmatic extension of the platform.

Technical Reviewers – Experts tasked with assessing the technical integrity and implementation quality of the codebase.

System Architects – Design stakeholders focused on validating architectural coherence and strategic technical choices.

Academic Assessors – Faculty members auditing the project for technical rigor and adherence to specified requirements.

1.3 System Overview

ConstructionGen solves a real-world problem faced by construction site managers: the time-consuming and error-prone process of writing daily site reports manually. The system accepts structured project metadata and a natural language description of site activities, then uses an LLM backed by a RAG pipeline to generate a professional, formatted report.

Core capabilities:

Feature	Description
Report Generation	AI generates structured Markdown reports from form inputs
Chat with AI	Domain-restricted Q&A assistant for construction topics
Report History	Persistent local storage of all generated reports
Export	Download reports as PDF, Word (.doc), or plain text
RAG Pipeline	ChromaDB vector store grounds responses in real construction knowledge

Technology Stack:

Layer	Technology
Frontend	React 18, TypeScript, Vite, React Router v6
Styling	Custom CSS with CSS Variables (no Tailwind)
Markdown Rendering	react-markdown + remark-gfm
Backend	Python 3, FastAPI, Uvicorn
LLM	Groq API — LLaMA 3.3 70B Versatile
LLM Orchestration	LangChain (langchain-groq)
Vector Store	ChromaDB (local persistent)

2. System Design

2.1 Architecture Diagram (ASCII)

```
┌──────────────────────────────────────────┐
│          BROWSER (Client)                │
│  ┌──────┬──────┬──────┬──────┐          │
│  │ Home │NewReport│ Chat │Hist │          │
│  └──────┴──────┴──────┴──────┘          │
│        React Router v6 + Layout          │
│        localStorage (reports/chat)       │
└──────────────────────────────────────────┘
                  │ HTTP POST
        /generate-report | /chat
                  │
                  ▼
┌──────────────────────────────────────────┐
│      BACKEND  FastAPI :8000              │
│  CORS → Routes → Pydantic                │
│  ┌──────────────────────────────┐        │
│  │       ConstructionRAG        │        │
│  │  ChromaDB ──► LangChain      │        │
│  │  HuggingFace Embeddings      │        │
│  │  MiniLM-L6-v2                │        │
│  └──────────────────────────────┘        │
└──────────────────────────────────────────┘
                  │ HTTPS
                  ▼
          ┌──────────────┐
          │   Groq Cloud │
          │ LLaMA 3.3 70B │
          └──────────────┘
```

2.2 UML Class Diagram

```
┌──────────────────────────────┐
│       ConstructionRAG        │
├──────────────────────────────┤
│ - llm : ChatGroq             │
│ - embeddings : HuggingFace   │
│ - vectorstore : Chroma       │
│ - system_prompt : str        │
├──────────────────────────────┤
│ + generate_report(...) : str │
│ + chat(message) : str        │
└──────────────────────────────┘
        │ uses
   ┌────┴────┐
   ▼         ▼
┌──────────┐  ┌──────────────────┐
│ ChatGroq │  │      Chroma      │
├──────────┤  ├──────────────────┤
│ model    │  │ persist_dir      │
│ temp=0.1 │  │ embedding_fn     │
│ invoke() │  │ similarity_search│
└──────────┘  └──────────────────┘
```

```
┌──────────────────┐   ┌──────────────────┐
│ ReportRequest    │   │ ReportResponse   │
├──────────────────┤   ├──────────────────┤
│ project_name     │   │ report : str     │
│ date, location   │   │ success : bool   │
│ topic, tone      │   │ message : str    │
│ length           │   └──────────────────┘
└──────────────────┘

┌──────────────────────┐
│   SavedReport (TS)   │
├──────────────────────┤
│ id, projectName      │
│ date, location       │
│ contractor, topic    │
│ content, createdAt   │
└──────────────────────┘
```

2.3 Sequence — Report Generation

```
User        NewReport      FastAPI       RAG          Groq
 |            |              |            |             |
 |—fill ——→|              |            |             |
 |—generate→|              |            |             |
 |            |—POST ——→|            |             |
 |            |              |—validate→|             |
 |            |              |            |—search——→|(ChromaDB)
 |            |              |            |←docs———|
 |            |              |            |—invoke()→|
 |            |              |            |←markdown—|
 |            |←response—|←content—|             |
 |            |—render+save→localStorage            |
 |←rendered—|              |            |             |
```

2.4 Sequence — Chat with AI

```
User        Chat.tsx       FastAPI       RAG          Groq
 |            |              |            |             |
 |—message→|              |            |             |
 |            |—POST /chat→|            |             |
 |            |              |—chat()→|             |
 |            |              |            |—search——→|(ChromaDB)
 |            |              |            |—invoke()→|
 |            |              |            |←response—|
 |            |←————————|←————|             |
 |            |—append+save→localStorage            |
 |←rendered—|              |            |             |
```

2.5 Activity — Report Generation

```
┌──────────────────┐
│ Open NewReport   │
└──────────────────┘
        │
        ▼
┌──────────────────┐
│   Fill form      │
└──────────────────┘
        │
        ▼
┌──────────────────┐   ┌──────────────────┐
│ topic empty?     │ —YES→ │ show error       │
└──────────────────┘   └──────────────────┘
        │ NO
        ▼
┌──────────────────┐
│ POST /generate   │
└──────────────────┘
        │
        ▼
┌──────────────────┐
│ Pydantic valid   │
└──────────────────┘
        │
        ▼
┌──────────────────┐
│ ChromaDB k=3     │
└──────────────────┘
```

```
        │
        ▼
┌──────────────────┐
│ Build prompt     │
│ tone + length    │
└──────────────────┘
        │
        ▼
┌──────────────────┐   ┌──────────────────┐
│ LLM success?     │ —NO→ │ show error       │
└──────────────────┘   └──────────────────┘
        │ YES
        ▼
┌──────────────────┐
│ render + save    │
│ localStorage     │
└──────────────────┘
        │
        ▼
┌──────────────────┐
│ Copy / Download  │
│ PDF, Word, TXT   │
└──────────────────┘
```

3. Application Design

3.1 Process Flow

The system follows a linear request-response flow with no server-side session state. All persistence is handled client-side via `localStorage`.

```
[User Input] → [Form Validation] → [HTTP POST] → [RAG Pipeline]
     → [LLM Generation] → [Markdown Response] → [Render + Save]
```

Report Generation Process:

User fills the form on **`NewReport.tsx`** with project metadata and site activity description

1. Client validates that the **`topic`** field is non-empty
2. A **`POST /generate-report`** request is sent with a **JSON** body
3. **FastAPI** validates the body against **`ReportRequest`** **Pydantic model**
4. **`ConstructionRAG.generate_report()`** performs a **vector similarity search** on **ChromaDB** using the topic as query
5. Top 3 matching document chunks are retrieved as context
6. A structured prompt is assembled with tone/length instructions, project info, context, and topic
7. The prompt is sent to **Groq's LLaMA 3.3 70B model** via **LangChain**
8. The **Markdown** response is returned to the frontend
9. **`ReportRenderer`** renders it using **`react-markdown`** + **`remark-gfm`**
10. The report is saved to **`localStorage`** via **`saveReport()`**

Chat Process:

1. User types a message in **`Chat.tsx`**
2. **`POST /chat`** is sent with **`{ message: string }`**
3. **`ConstructionRAG.chat()`** retrieves 3 relevant context chunks from ChromaDB
4. A grounded prompt is built with strict "do not fabricate" instructions
5. LLM response is returned and rendered via **`ReactMarkdown`**
6. Full conversation is persisted in **`localStorage`**

3.2 Information Flow

```
┌──────────────────────────────────────────────────────────────────┐
│                        INFORMATION FLOW                          │
│                                                                  │
│  Form Fields (projectName, date, location, etc.)                 │
│       │                                                          │
│       ▼                                                          │
│  ReportRequest JSON payload                                      │
│       │                                                          │
│       ▼                                                          │
│  FastAPI → ConstructionRAG.generate_report()                     │
│       │                                                          │
│       ├──► ChromaDB.similarity_search(topic, k=3)                │
│       │          └──► Returns: List[Document]                    │
│       │                                                          │
│       ├──► PromptTemplate.format(topic, context)                 │
│       │          └──► Returns: str (full prompt)                 │
│       │                                                          │
│       ├──► ChatGroq.invoke(prompt)                               │
│       │          └──► Returns: AIMessage.content (Markdown)      │
│       │                                                          │
│       ▼                                                          │
│  ReportResponse { report, success, message }                     │
│       │                                                          │
│       ▼                                                          │
│  Frontend: setGeneratedReport(data.report)                       │
│       │                                                          │
│       ├──► ReportRenderer (react-markdown display)               │
│       └──► saveReport() → localStorage                           │
└──────────────────────────────────────────────────────────────────┘
```

3.3 Information Flow

Frontend Components

Component	File	Responsibility
`App`	`App.tsx`	Root router, wraps all pages in `Layout`
`Layout`	`Layout.tsx`	Renders `Sidebar` + `main-content` wrapper
`Sidebar`	`Sidebar.tsx`	Fixed floating nav dock with 4 route links
`Header`	`Header.tsx`	Top bar with app name and branding
`ReportRenderer`	`ReportRenderer.tsx`	Renders Markdown string via react-markdown + remark-gfm
`Home`	`pages/Home.tsx`	Landing page with hero section and 3 feature cards
`NewReport`	`pages/NewReport.tsx`	Report generation form, progress bar, output display
`Chat`	`pages/Chat.tsx`	Chat interface with message history, suggestions, clear button
`History`	`pages/History.tsx`	Report list with search, view, download, delete

Backend Components

Component	File	Responsibility
`FastAPI app`	`main.py`	HTTP server, CORS, route definitions, Pydantic models
`ConstructionRAG`	`rag.py`	RAG pipeline: embeddings, vector search, prompt building, LLM call
`ChromaDB`	`chroma_db/`	Local persistent vector store for construction knowledge
`HuggingFace Embeddings`	`rag.py`	Converts text to vectors using `all-MiniLM-L6-v2`
`ChatGroq`	`rag.py`	LangChain wrapper for Groq API calls
`ingest_docs.py`	`ingest_docs.py`	One-time script to embed and store construction documents

Storage Layer

Store	Key	Contents
`localStorage`	`construction_reports`	Array of `SavedReport` objects
`localStorage`	`chatMessages`	Array of `Message` objects for chat history
`chroma_db/`	SQLite + bin files	Embedded construction document vectors

4. LLM Selection & Rationale

Selected Model: LLaMA 3.3 70B Versatile via Groq API

Property	Value
Model	`llama-3.3-70b-versatile`
Provider	Groq Cloud
Temperature	`0.1`
Max Tokens	`4096`
Orchestration	LangChain (`langchain-groq`)

Rationale

Why LLaMA 3.3 70B?

- Size and capability: At 70 billion parameters, LLaMA 3.3 provides near-GPT-4-level reasoning and instruction following, which is essential for generating structured, multi-section professional reports
- Instruction adherence: The model reliably follows complex formatting instructions (Markdown tables, section headings, bullet points) without deviation

- Low hallucination at low temperature: At `temperature=0.1`, the model stays close to the provided context and input data, minimising fabrication of project details

Why Groq?

- Inference speed: Groq's LPU (Language Processing Unit) hardware delivers significantly faster token generation than standard GPU-based providers, resulting in sub-5-second report generation
- Free tier availability: Groq provides a generous free API tier suitable for development and demonstration
- LangChain integration: `langchain-groq` provides a drop-in replacement for other LangChain LLM providers, making the integration clean and maintainable

Why not GPT-4 or Gemini?

Criterion	GPT-4	Gemini 2.5 Flash	LLaMA 3.3 70B (Groq)
Speed	Moderate	Fast	Very Fast
Cost	High	Moderate	Free tier available
Hallucination control	Good	Moderate	Good at low temp
Open source	No	No	Yes (Meta)
LangChain support	Yes	Yes	Yes

5. Approach Toward Solving the Problem Statement

Problem Statement

Construction site managers spend significant time writing daily site reports manually. These reports must be professional, structured, accurate, and consistent — yet the process is repetitive and error-prone when done by hand.

Solution Approach

The solution is built around three core design principles:

1. Structured Input → Grounded Output

Rather than asking the LLM to generate a report from a vague prompt, the system collects structured metadata (project name, date, location, contractor, weather) and combines it with a detailed site activity description. This structured input is embedded directly into the prompt, preventing the model from inventing project details.

2. RAG for Domain Grounding

A Retrieval-Augmented Generation pipeline using ChromaDB ensures that the LLM's responses are grounded in real construction knowledge. When a report topic is submitted, the top 3 most semantically similar document chunks are retrieved and injected into the prompt as context. This means the model can reference actual construction standards, safety procedures, and best practices rather than relying solely on its training data.

3. Tone and Length as Explicit Instructions

Rather than passing `tone=formal` as a label and hoping the model interprets it correctly, the system maps each option to a concrete instruction:

- `formal` → "Write in third person. Avoid contractions. Use precise technical terminology suitable for client or regulatory submission."
- `brief` → "Total report should be under 300 words. Only include the most critical information."

This ensures the user's selection has a measurable, consistent effect on the output.

4. Client-Side Persistence

Reports are saved to `localStorage` immediately after generation. This eliminates the need for a database, reduces backend complexity, and ensures the user's data is always available offline. The History page provides full CRUD operations (view, download, delete) on saved reports.

5. Multi-Format Export

Reports are exported in three formats to serve different professional needs:

- PDF — via jsPDF, with proper title, divider, and auto page breaks
- Word (.doc) — styled HTML wrapped in a Word-compatible document
- TXT — plain text with all Markdown symbols stripped

6. References

- FastAPI Documentation: https://fastapi.tiangolo.com
- LangChain Documentation: https://python.langchain.com
- LangChain Groq Integration: https://python.langchain.com/docs/integrations/chat/groq
- Groq API Documentation: https://console.groq.com/docs
- Meta LLaMA 3.3 Model Card: https://huggingface.co/meta-llama/Llama-3.3-70B-Instruct
- ChromaDB Documentation: https://docs.trychroma.com
- HuggingFace sentence-transformers: https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2
- React Documentation: https://react.dev
- React Router v6: https://reactrouter.com
- react-markdown: https://github.com/remarkjs/react-markdown
- remark-gfm: https://github.com/remarkjs/remark-gfm
- jsPDF: https://github.com/parallax/jsPDF
- Vite: https://vitejs.dev